ScoreFormer training: ScoreFormer.Train

Input: D: molecules and docking scores; H: model and optimization settings, including weighting strength α and log-weight cap c; E: maximum training epochs.

Output: f_θ: trained ScoreFormer docking-score surrogate.

Prepare molecular representations G from D for ScoreFormer.
Split G into training set G_{tr} and validation set G_{val}.
Initialize ScoreFormer parameters θ, optimizer and learning-rate schedule using H.
for $e = 1, \ldots, E$ **do**
    Set f_θ to training mode.
    for each mini-batch $B \subseteq G_{tr}$ **do**
        $\hat{y}_i \leftarrow f_\theta(G_i)$ for each molecule $i \in B$.
        $w_i \leftarrow \exp[\min(-\alpha y_i, c)]$.
        $\mathcal{L}_B \leftarrow (1 / |B|) \sum_{i \in B} w_i(\hat{y}_i - y_i)^2$.
        Update θ using the gradient $\nabla_\theta \mathcal{L}_B$ and the optimizer.
    end for
    Set f_θ to evaluation mode.
    Compute validation loss and prediction metrics on G_{val}.
    Save a checkpoint according to the configured policy.
    Update the learning-rate schedule.
    if a configured stopping condition is met **then break**.
end for
return f_θ.

ScoreFormer inference: ScoreFormer.Infer

Input: L: compounds to be scored; f_θ: trained model returned by ScoreFormer.Train.

Output: $\hat{\mathrm{Y}}$: predicted docking scores associated with compound identifiers.

Set f_θ to evaluation mode; disable gradients and dropout.
$\hat{\mathrm{Y}} \leftarrow$ empty collection.
for each molecular mini-batch B in L **do**
    $G_\mathrm{B} \leftarrow$ prepare ScoreFormer molecular representations for B.
    $\hat{\mathrm{y}}_\mathrm{B} \leftarrow f_\theta(G_\mathrm{B})$.
    Append compound identifiers and $\hat{\mathrm{y}}_\mathrm{B}$ to $\hat{\mathrm{Y}}$.
end for
return $\hat{\mathrm{Y}}$.

ALScreen: active-learning framework

Input: L: compound library; O_T: docking oracle for target T; n_{sub}: initial subset size; n_{oracle}: compounds docked per cycle; n_{pool}: labeled compounds added per cycle; K: maximum cycles; H, E: training settings.

Output: R: ranked list of prioritized compounds.

$S \leftarrow$ RepresentativeSubset(L, n_{sub}).
$D \leftarrow O_T(S)$.
$P \leftarrow D$. // Initialize labeled training pool.
$f_\theta \leftarrow$ ScoreFormer.Train(P, H, E). // ScoreFormer initial training.
$U \leftarrow L \setminus S$. // Compounds not yet docked.
for $k = 1, \ldots, K$ **do**
    if U is empty **then break**.
    $\hat{Y} \leftarrow$ ScoreFormer.Infer(U, f_θ).
    $G \leftarrow$ TopRanked($\hat{Y}$, n_{oracle}). // Lowest predicted scores.
    $D_G \leftarrow O_T(G)$.
    $e_i \leftarrow |\hat{y}_i - y_i|$ for each compound i in D_G.
    $C \leftarrow$ select n_{pool} labeled records from D_G, split equally
        between the lowest and highest absolute prediction errors.
    $P \leftarrow P \cup C$. // Retain compound identities and docking labels.
    $U \leftarrow U \setminus G$.
    $f_\theta \leftarrow$ ScoreFormer.Train(P, H, E). // ScoreFormer retraining.
    if the configured scaffold-convergence criterion is met **then break**.
end for
$R \leftarrow$ RankAscending(ScoreFormer.Infer(L, f_θ)).
return R.